

PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

SOFTWARE ENGINEERING – MINI PROJECT

Project: SmartSpend — Smart Expense & Budget Management Application

Software Requirements Specification

for SmartSpend (IEEE 830 Format)
Version 1.0

Team H5:

Sriram S – PES1UG24CS469

Sriram K – PES1UG24CS468

Srijan Kumar – PES1UG24CS467

Sri Pranav Gautam Buduguru – PES1UG24CS466

Table of Contents

1. Introduction
 2. Overall Description
 3. External Interface Requirements
 4. Analysis Models (Use Case Diagram)
 5. System Features (Functional Requirements)
 6. Other Nonfunctional Requirements
 7. Security Requirements
 8. Other Requirements
- Appendix A: Glossary
- Appendix B: Field Layouts
- Appendix C: Requirement Traceability Matrix

1. Introduction

1.1 Purpose

This document specifies the complete software requirements for Version 1.0 of SmartSpend, a web-based expense tracking and budget management application developed by Team H5 for the Software Engineering (UE24CS341A) mini-project. It follows the standard IEEE 830 SRS structure. This SRS covers the entire system as a single, self-contained product; there are no external subsystems in scope.

1.2 Intended Audience and Reading Suggestions

This SRS is intended for four audiences: (1) the development team, who should read Sections 2 through 7 in full before implementation begins; (2) the course instructor/evaluator, who may focus on Sections 5, 7, and Appendix C for functional and security completeness and traceability; (3) any future maintainer of the system, who should treat this document as the baseline specification; and (4) testers, who will derive the Test Plan directly from Sections 5, 6, and 7. The document proceeds from general context (Section 2) to specific, itemized, individually verifiable requirements (Sections 5–7).

1.3 Product Scope

SmartSpend allows an individual user to record expenses, organize them by category, define spending budgets, receive automated alerts when spending nears or exceeds a budget threshold, and view analytics and reports summarizing financial activity over time. An administrator manages the master category list and has visibility into aggregate, anonymized system usage. The product's objective is to give users real-time visibility into personal spending habits without requiring integration with real banking infrastructure, keeping the system self-contained and achievable within a single academic semester.

1.4 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.

2. Overall Description

2.1 Product Perspective

SmartSpend is a new, self-contained web application; it is not a follow-on to, or replacement for, any existing system. It exposes a REST API consumed by a browser-based front end. The overall system context is: User/Admin (via browser) → Web UI → REST API → Service Layer → Database. No third-party financial or banking system is integrated in this version — budgets and expenses are entirely user-declared.

2.2 Product Functions

At a high level, the product performs the following functions (each is elaborated with individual requirements in Section 5):

- Account registration and authentication.
- Expense entry, editing, deletion, and categorization.
- Budget creation and threshold tracking per category and period.
- Automated alert generation when a budget threshold is breached.
- Analytics dashboard and exportable reports.
- Administrative management of master categories and aggregate system oversight.

2.3 User Classes and Characteristics

User Class	Privileges	Characteristics
User	Create/manage own expenses and budgets; view own analytics and alerts	Non-technical; frequent, short sessions; primary user class
Administrator	Manage master categories; view aggregate, anonymized system reports	Higher privilege; infrequent, purposeful sessions; technical/trained

2.4 Operating Environment

Server: Linux-based host running the application (Python/Django), containerized with Docker.

Client: Any modern web browser (Chrome, Firefox, Edge) on desktop or mobile viewport.

Database: PostgreSQL/MySQL

2.5 Design and Implementation Constraints

- Must be implemented using either Python/Django or JavaScript (MERN stack).
- Must use Git/GitHub for version control, with CI/CD via Jenkins or an equivalent tool.
- Must follow Agile methodology with visible, ongoing sprint/backlog activity in GitHub Projects or Jira.
- Passwords and other sensitive fields must never be stored in plaintext.
- All client-server communication must occur over HTTPS.

2.6 Assumptions and Dependencies

- Assumes users manually enter expenses; no live bank-feed integration is in scope for this version.
- Assumes a single-currency deployment for the academic version of the product.
- Depends on no external payment/bank verification — all amounts are user-declared and are not reconciled against a financial institution.
- Assumes availability of a Docker-capable host for deployment and CI/CD demonstration.

3. External Interface Requirements

3.1 User Interfaces

The system provides a web-based UI comprising: a login/registration screen, an expense entry form, a category management view, a budget configuration screen, an analytics dashboard, an alerts panel, and an admin panel (visible only to Administrator accounts). Standard UI conventions apply throughout: consistent primary navigation, in-line validation error messages, and a persistent indicator for any active budget alerts.

3.2 Software Interfaces

The front end communicates with the back end exclusively via a REST API using JSON payloads. The back end communicates with the database through an ORM (Django ORM) abstraction layer; no component accesses the database directly other than through this layer.

3.3 Communications Interfaces

All client-server traffic uses HTTPS. Budget-breach alerts are surfaced in-application on dashboard/login load; email or push notification is explicitly out of scope for this version and is noted as a possible future enhancement.

4. Analysis Models

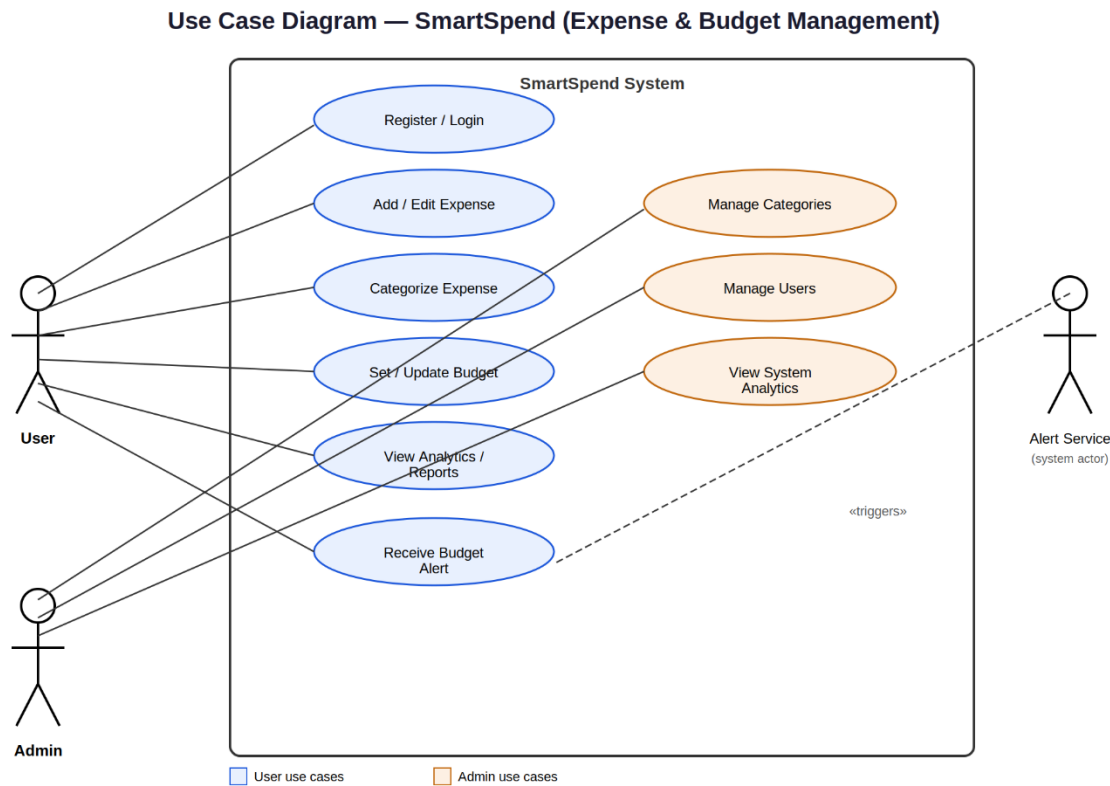
4.1 Actors

- User — the primary actor; registers, logs in, manages expenses and budgets, views analytics, and receives alerts.
- Administrator — manages the master category list and monitors aggregate, system-level analytics.
- Alert Service — an internal system actor representing the automated component that evaluates budget thresholds and triggers alerts without direct human initiation.

4.2 Use Cases Grouped by Actor

Actor	Use Cases
User	Register/Login; Add/Edit Expense; Categorize Expense; Set/Update Budget; View Analytics/Reports; Receive Budget Alert
Administrator	Manage Categories; Manage Users; View System Analytics
Alert Service	Triggers “Receive Budget Alert” for the User when a threshold condition is met

4.3 Use Case Diagram



5. System Features (Functional Requirements)

5.1 Feature: User Registration & Authentication

Description: High priority. Allows a user to create an account and securely authenticate before any personal financial data is accessible.

Req ID	Requirement	Priority	Verification
FR-001	The system shall allow a new user to register using a unique email address and a password of at least 8 characters.	High	Functional test
FR-002	The system shall reject a registration attempt if the supplied email address already exists in the system, returning an unambiguous error message.	High	Negative test
FR-003	The system shall authenticate a registered user by verifying email and password before granting access to any protected resource.	High	Functional test
FR-004	The system shall issue a session token valid for no more than 24 hours upon successful authentication.	Medium	Functional test

5.2 Feature: Expense Entry & Categorization

Description: High priority. Core feature — lets a user log day-to-day spending against a category.

Req ID	Requirement	Priority	Verification
FR-005	The system shall allow an authenticated user to add an expense record with amount, category, date, and an optional note of up to 255 characters.	High	Functional test
FR-006	The system shall allow a user to edit or delete only expense records they own.	High	Functional + authorization test
FR-007	The system shall allow a user to select a category from a predefined list or create a custom category limited to 50 characters.	Medium	Functional test
FR-008	The system shall reject an expense entry with an amount less than or equal to zero, returning a validation error.	High	Negative test

5.3 Feature: Budget Management

Description: High priority. Lets a user define spending limits per category and time period.

Req ID	Requirement	Priority	Verification
FR-009	The system shall allow a user to create a budget for a given category and period (weekly or monthly) with a positive limit amount.	High	Functional test
FR-010	The system shall allow a user to update or delete a budget they own.	Medium	Functional test
FR-011	The system shall recalculate cumulative spend against each active budget within 1 second of a new expense being saved to that category.	High	Unit + system test

Req ID	Requirement	Priority	Verification
FR-012	The system shall prevent two active budgets from being created for the same category and overlapping period.	Medium	Negative test

5.4 Feature: Alerts

Description: High priority. Automated, system-triggered notifications when spend approaches or breaches a budget.

Req ID	Requirement	Priority	Verification
FR-013	The system shall generate an alert the first time cumulative spend in a category reaches 80% of its budget for the active period.	High	System test
FR-014	The system shall generate a second, distinct alert the first time cumulative spend in a category exceeds 100% of its budget.	High	System test
FR-015	The system shall display all active, unacknowledged alerts to the user immediately upon login or dashboard load.	Medium	Functional test

5.5 Feature: Analytics & Reporting

Description: Medium priority. Gives the user visibility into spending trends.

Req ID	Requirement	Priority	Verification
FR-016	The system shall display a breakdown of spend by category for a user-selected date range.	Medium	Functional test
FR-017	The system shall display a spend trend chart (daily or weekly granularity) for a user-selected date range.	Medium	Functional test
FR-018	The system shall allow a user to export a summary report of their expenses for a selected period in CSV format.	Low	Functional test

5.6 Feature: Administration

Description: Medium priority. Administrative oversight of shared, system-wide data.

Req ID	Requirement	Priority	Verification
FR-019	The system shall allow an administrator to add, edit, or disable master expense categories.	Medium	Functional test
FR-020	The system shall allow an administrator to view aggregate, anonymized system usage analytics (e.g., total active users, total categories in use).	Low	Functional test
FR-021	The system shall prevent a non-administrator account from invoking any administrative endpoint, returning HTTP 403.	High	Authorization test

5.7 Stimulus/Response Sequence — Representative Example

For the “Add Expense” feature (FR-005 through FR-008): the user submits an expense form (stimulus) → the system validates the input against FR-008 → on success, the system persists the record and re-evaluates the relevant budget (FR-011) → the system responds with a success confirmation and, where applicable, an alert notice (FR-013/FR-014). This corresponds directly to Sequence Diagram 1 in the Architecture & Design Specification.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

ID	Measurable Requirement	Verification
NFR-001	95% of dashboard/analytics requests shall receive a response within 2 seconds under a defined test load of 50 concurrent users.	Performance test
NFR-002	Budget threshold evaluation shall complete within 1 second of a new expense being saved (see FR-011).	Performance test

6.2 Safety Requirements

SmartSpend has no physical or life-safety-critical component. The system shall, however, never silently drop or lose a successfully submitted expense record; any persistence failure shall be surfaced to the user as an explicit error rather than a silent no-op.

6.3 Security Requirements

See Section 7 for the complete, standalone Security Requirements section.

6.4 Software Quality Attributes

- Maintainability: business logic shall be separated into identifiable, independently testable modules (see Section 6.5 of the Architecture & Design Specification).
- Usability: a new user shall be able to log a basic expense in no more than 3 screens/steps.
- Reliability: an expense shall not be duplicated if a user retries submission after a failed or timed-out save.
- Testability: core services shall expose interfaces that are testable independently of the UI layer, using mocks for external dependencies.

6.5 Business Rules

- Only categories marked active can be assigned to a new expense.
- A budget period cannot overlap with another active budget for the same category (see FR-012).
- Only an administrator can disable a category; disabling a category does not delete historical expenses already tied to it.
- An expense record must always be associated with exactly one category.

7. Security Requirements

7.1 Security Objectives

The following security objectives govern the design and validation of SmartSpend, in line with standard confidentiality/integrity/availability (CIA) goals adapted to this system's scope:

1. Confidentiality: a user's financial data (expenses, budgets, personal profile) shall be accessible only to that user and, where explicitly scoped, to an authenticated administrator performing an authorized administrative function — never to any other user or unauthenticated party.
2. Integrity: expense and budget records shall be modifiable only by their owning user (or a properly authorized administrative process), and all state-changing requests shall be validated server-side so that no client can bypass business rules (e.g., FR-008, FR-012) by manipulating a request directly.
3. Accountability: security-relevant events (failed logins, unauthorized access attempts, administrative actions) shall be logged with sufficient detail to support after-the-fact review, without ever logging plaintext credentials.

7.2 Security Requirements

ID	Security Requirement	Validation Approach
SEC-001	Passwords shall be stored using a salted, one-way cryptographic hash (e.g., bcrypt/argon2); they shall never be stored or logged in plaintext.	Code/configuration review
SEC-002	Role-based access control (RBAC) shall prevent a regular User account from accessing any Administrator-only endpoint or function.	Authorization tests
SEC-003	The system shall validate and sanitize all user-supplied input on the server side before processing, rejecting malformed or unexpected input.	Negative/security tests
SEC-004	Authenticated API endpoints shall reject requests carrying an expired, malformed, or otherwise invalid session token, returning HTTP 401.	Security test
SEC-005	Failed login attempts and unauthorized access attempts (e.g., a User calling an Admin endpoint) shall be logged with timestamp, account identifier, and endpoint, without recording the attempted password.	System test
SEC-006	Sensitive fields (password hash, session tokens) shall never be included in any API response body.	Code review + API test

7.3 Traceability Note

Security objectives 7.1(1)–(3) map to Security Requirements SEC-001 through SEC-006 as follows: Confidentiality → SEC-001, SEC-002, SEC-006; Integrity → SEC-002, SEC-003; Accountability → SEC-005. This mapping is carried forward into the Test Plan's Section 5.1 (Security Validation) and into Appendix C below.

8. Other Requirements

Data retention: expense and budget history shall be retained indefinitely unless the user deletes their account, at which point all associated personal data shall be purged. Localization and multi-currency support are explicitly out of scope for this academic version and are noted as potential future enhancements.

Appendix A: Glossary

Term	Definition
Expense	A single recorded instance of spending, with amount, category, date, and optional note.
Category	A label used to classify expenses (e.g., Food, Transport, Utilities).
Budget	A user-defined spending limit for a category over a time period (weekly or monthly).
Threshold Alert	A system-generated notification triggered when cumulative spend nears or exceeds a budget (see FR-013/FR-014).
RBAC	Role-Based Access Control — restricting functionality based on user role (User vs Administrator).
Session Token (JWT)	A signed token issued at login and used to authenticate subsequent API requests.

Appendix B: Field Layouts

Field layout for the core Expense entity (illustrative; the complete layout for all entities appears in the Architecture & Design Specification, Section on Database Design):

Field	Length	Data Type	Description	Mandatory
expense_id	36	UUID/String	Unique expense identifier	Y
user_id	36	UUID/String	Owning user reference	Y
category_id	36	UUID/String	Associated category reference	Y
amount	10,2	Decimal	Expense amount (must be > 0, per FR-008)	Y
expense_date	8	Date	Date the expense occurred	Y
note	255	String	Optional free-text note	N

Appendix C: Requirement Traceability Matrix (RTM)

This matrix traces each requirement forward to its architecture reference, design reference, code file reference, and test case IDs. Architecture and design references are populated from the accompanying Architecture & Design Specification; test case IDs are populated from the accompanying Test Plan. Code references remain TBD until implementation (Phase 3).

Req ID	Brief Description	Architecture Ref	Design Ref	Code Ref	Test Case ID
FR-001	User registration	Auth Service (3.1)	Seq. Diagram 2	TBD	TC-FUN-01
FR-003	User authentication	Auth Service (3.1)	Seq. Diagram 2	TBD	TC-FUN-02

Req ID	Brief Description	Architecture Ref	Design Ref	Code Ref	Test Case ID
FR-005	Add expense entry	Expense Service (3.1)	Seq. Diagram 1	TBD	TC-FUN-03
FR-008	Reject non-positive amount	Expense Service (3.1)	Seq. Diagram 1	TBD	TC-FUN-04
FR-009	Create budget	Budget Service (3.1)	Class design (4.1)	TBD	TC-FUN-05
FR-011	Real-time threshold recalculation	Alert Engine (3.1)	Seq. Diagram 1	TBD	TC-NFR-01
FR-013	Alert at 80% threshold	Alert Engine (3.1)	Seq. Diagram 1	TBD	TC-FUN-06
FR-016	Category-wise spend breakdown	Report Service (3.1)	API Design (4.2)	TBD	TC-FUN-07
FR-019	Admin manages categories	Admin/API layer (3.1)	API Design (4.2)	TBD	TC-FUN-08
SEC-001	Password hashing	Auth Service (3.1)	Seq. Diagram 2	TBD	TC-SEC-01
SEC-002	RBAC enforcement	API/Controller layer (3.1)	API Design (4.2)	TBD	TC-SEC-02
SEC-004	Reject invalid session token	API/Controller layer (3.1)	API Design (4.2)	TBD	TC-SEC-03
NFR-001	Dashboard response time	Full stack (3.1)	Section 6.4 (perf.)	TBD	TC-NFR-02